

Natural Language Processing



A Review-Grounded NLP System for Product Recommendation and Review Summarization in Beauty Products

Anushka Thagle (axt5884@psu.edu)

Supanut Chindawan (sxc6473@psu.edu)

School of Graduate Professional Studies

MPS/MS in Data Analytics

March, 2026

Document Control

Work carried out by:

Name	Email Address	Other
Anushka Thagle	axt5884@psu.edu	
Supanut Chindawan	sxc6473@psu.edu	

Revision Sheet

Release No.	Date	Revision Description
1	2026-03-22	Define Project Description, Challenges, and available solution.
2	2026-03-28	Literature Review
3	2026-04-03	Data Description
4	2026-04-11	Exploratory Data Analysis and Model Architecture Proposed
5	2026-04-19	Training/Testing strategy and Evaluation metrics
6	2026-04-26	Obtained results Conclusion of your study Codebase and instructions to run your application
7	2026-04-30	Feedback Addressed Limitation and Future Work Use Case and Application

Note: For the project assignment deadline submission please refer to the syllabus. Each week's assignment is for 10 points and the project assignment is worth 40% of your final grade. The project presentation is worth 5% of your final grade. The course project should be completed as a team with no more than 3 members (in each team).

TABLE OF CONTENTS

Document Control.....	1
Work carried out by:	2
Revision Sheet.....	2
TABLE OF CONTENTS.....	2
Project Description	4
Challenges.....	5
Is there any available solution to this problem?.....	5
Literature Review	5
Paper 1: Bridging Language and Items for Retrieval and Recommendation	6
Paper 2: Accurate and Explainable Recommendation via Review Rationalization (R3)	6
Paper 3: Learning to Ask Appropriate Questions in Conversational Recommendation	7
Paper 4: QQSUM: A Novel Task and Model of Quantitative Query-Focused Summarization for Review-based Product Question Answering	8
Paper 5: Disambiguation in Conversational Question Answering in the Era of LLM: A Survey	9
Paper 6: Literature Survey Entry: A hallucination detection and mitigation framework for faithful text summarization using LLMs (Liu et al., 2025, Scientific Reports)	10
How Surveyed Literature Guides the Project	10
Data Description.....	11
Dataset Subset Selection	12
Token Definitions	12
Data Files and Structure	12
User Reviews	13
Example of User Reviews dataset (json format):	13
Item Metadata.....	14
Data Visualization	16
Rating Distribution	17
The Number of Reviews Over Time	17
Review Text Length Distribution	17
Word Cloud.....	18
Deep learning Process.....	19

Offline Phase: Sentiment-Aware Data Ingestion	20
Data Preprocessing.....	21
Text Encoder: BLAIR	21
Sentiment Classifier: CNN	21
Vector Database: Metadata-Augmented Storage.....	21
Online Phase: Query Pipeline.....	22
Sub-task 1: Clarification Check	22
Sub-task 2: Sentiment-Aware Retrieval	22
Sub-task 3: Evidence-Grounded Summarization.....	22
<i>Training and Testing Strategy</i>	<i>23</i>
Sentiment Classifier (CNN).....	23
Data Source and Labeling Strategy	23
Data Split	23
Class Imbalance Handling.....	23
Hyperparameter Tuning.....	23
Training Procedure	23
Confidence Threshold Calibration.....	24
Clarification Classifier (BiLSTM).....	24
Data Source and Labeling Strategy	24
Data Split	24
Hyperparameter Tuning.....	24
Training Procedure	24
Retrieval Module (BLAIR + Vector Database).....	24
Embedding Strategy	24
Retrieval Configuration	25
Ground Truth Construction	25
Extractive Summarizer (BiLSTM)	25
Data Source and Labeling Strategy	25
Data Split	25
Hyperparameter Tuning.....	25
Training Procedure.....	25
<i>Evaluation Metrics.....</i>	<i>26</i>
Sentiment Classifier Evaluation.....	26
Clarification Classifier Evaluation	26
Retrieval Evaluation	26
Summarization Evaluation	26
Automatic Metrics.....	26
Faithfulness Evaluation	26
<i>Obtained results</i>	<i>27</i>
Phase 0-1: Setup.....	27
Phase 2: Dataset Statistics and Preprocessing Outcomes	27

Phase 3: Train CNN Sentiment Classifier	27
Build vocabulary and Create Embedding	27
CNN Sentiment Classifier	27
Training Curves Analysis	28
Test Set Evaluation	29
Confidence Threshold Calibration	30
Qualitative Sanity Check	32
Phase 4: BLAIR Encoding + FAISS Vector Database	32
Run CNN Sentiment Classifier on All Sentences	33
FAISS Indexing	33
Phase 5: BiLSTM Clarification Classifier	33
Dataset Construction	33
5-Fold Cross-Validation Results	34
Training Loss Convergence	34
Test Set Evaluation	34
Analysis of Asymmetric Precision–Recall:	35
Phase 6: BiLSTM Extractive Summarizer	35
Importance Label Construction	35
Training Results	35
ROUGE Evaluation	36
Phase 7: Retrieval Evaluation — BLAIR vs. Baselines	37
Ground Truth Construction	37
Retrieval Methods Compared	37
Metric Explanations and Analysis	37
Phase 8: End-to-End Pipeline Assembly	38
Phase 9: Faithfulness Evaluation	39
Method	39
Results	39
<i>Final Pipeline Results Summary</i>	<i>40</i>
<i>Conclusion of the study</i>	<i>40</i>
<i>Limitations</i>	<i>41</i>
<i>Future Work</i>	<i>42</i>
<i>Use Cases and Applications</i>	<i>42</i>
Use Case 1: Consumer-facing shopping assistance.	43
Use case2: Enterprise product feedback analysis.	43
<i>Reference</i>	<i>43</i>
<i>Appendix</i>	<i>44</i>
Evaluation Metrics	49
Code Base	49

Instruction to Run the code49

TABLE OF TABLES

Table 1 Statistics summary of the Amazon Reviews 2023	14
Table 2 Statistic summary of All_Beauty dataset.....	14
Table 3 the details of User Reviews file.....	15
Table 4 The details of Item Metadata	17
Table 5: Vector types stored in the vector database	23
Table 6: Structure of each record in the review vector collection	24
Table 7: CNN Sentiment Classifier Architecture	30
Table 8: CNN Sentiment Classifier Sanity Check.....	33
Table 9: Sentiment Distribution After CNN Inference.....	34
Table 10: Clarification question data.....	35
Table 11: Retrieval evaluation results	38
Table 12: Results summary 1	41
Table 13: Results summary 2	41
Table 14 : Evaluation metrics for each component.....	48

TABLE OF FIGURES

Figure 1 Rating Distribution of all Beauty reviews.....	19
Figure 2 The monthly review volume of Amazon Beauty products from 2000 to 2023.....	19
Figure 3: Distribution of review text length in word count (left) and Boxplot of text length by star rating (right)	20
Figure 4: The word cloud of Amazon Beauty product reviews.....	21
Figure 5 The overview system diagram.....	22

Project Description

Online beauty products receive extremely large volumes of user-generated reviews, making it difficult for shoppers to (a) identify which products match their needs and (b) understand the real tradeoffs discussed across thousands of reviews. This project frames the problem as an end-to-end NLP system for review-grounded product recommendation and summarization. Given a user query (e.g., “recommend a shampoo”), the system will recommend a small set of relevant products based on review content, and for each recommendation generate a concise pros-and-cons review summary. The key novelty is that summaries are evidence-grounded and citation-first: every pro or con must be supported by explicit review excerpts, and the system must not fabricate cons (or pros) that are not present in the evidence. In addition, when a user query is overly generic (e.g., “recommend a shampoo”), the system will ask 2–3 targeted clarification questions (such as hair/scalp type, primary concern, and budget preference) to refine recommendations and produce more relevant review-grounded summaries.

Challenges

There are 3 main challenges in our project:

1. **Evidence faithfulness / hallucination control:**

Generative models often produce plausible pros/cons even when the source text does not support them. A major challenge is enforcing that each summary claim is grounded in retrieved review evidence, and that the system abstains from adding cons when reviews do not mention them.

2. **Retrieval quality and coverage:**

Review corpora are large, and the system’s outputs depend heavily on retrieving representative evidence. If retrieval misses key negative experiences, the system may incorrectly present a product as having “no cons.” Conversely, retrieval may over-surface rare complaints, making a product look worse than the overall consensus.

3. **Clarification strategy for generic queries:**

Asking follow-up questions can improve relevance, but too many questions degrade user experience. The challenge is designing a small set of high-impact clarifying questions that meaningfully reduce ambiguity (e.g., hair type, concern, budget) while keeping the interaction lightweight and consistent.

Is there any available solution to this problem?

There are existing solutions for product recommendation and review summarization in e-commerce, including systems that rank products by ratings, keywords, or general relevance, and tools that generate generic summaries of reviews. However, many available solutions do not reliably guarantee evidence-grounded outputs: they may generate pros/cons that are not explicitly supported by reviews, or present balanced “pros and cons” even when one side is not actually present in the retrieved evidence. In addition, generic recommenders often perform poorly when the user’s query is underspecified (“recommend a shampoo”) because they do not systematically elicit preferences before recommending products. This project builds on existing approaches while addressing these gaps by focusing on citation-first, evidence-grounded pros/cons summaries and lightweight clarification-driven recommendation refinement, which directly targets trustworthiness and usefulness for real users.

Literature Review

Paper 1: Bridging Language and Items for Retrieval and Recommendation

Hou et al. (2024) present BLAIR, a series of pre-trained sentence embedding models designed specifically for recommendation and retrieval settings. The objective of the project is to bridge natural language and items in a shared embedding space. So, the model can better understand product-related text, such as reviews, queries, and item metadata. Moreover, the authors released AMAZON REVIEWS 2023, a large-scale and updated dataset containing over 571 million reviews and 48 million items across 33 categories.

What work has been pursued

The paper focuses on building a general-purpose text embedding model for recommendation scenarios. The study aims to learn the relationship between user review text and item metadata rather than relying only on keyword match. The authors also evaluate the model on three tasks: (a) sequential recommendation, (b) conventional product search, and (c) complex product search. The last task uses the input as a long, detailed natural language description rather than a short keyword query.

Tools and techniques

BLAIR is built on a RoBERTa-based sentence embedding architecture. The model uses the hidden layers of the CLS token as the sentence embedding and applies contrastive learning to align pairs of language context and item metadata. The training objective combines a contrastive loss with the original pretraining objective of the backbone model, namely masked language modeling (MLM). This design allows the model to learn both general language patterns and direct language item correspondence.

Metrics

For sequential recommendation, the paper reports Recall@10, NDCG@10, Recall@50, and NDCG@50. For product search and complex product search, the paper uses NDCG@100 as evaluation metric because they measure how highly the correct item is ranked among candidate products.

Result

BLAIR outperforms the general-purpose baseline, especially in language-heavy retrieval tasks. In sequential recommendation, text-based methods using BLAIR perform better than ID-based methods, showing that BLAIR provides strong item representations. In conventional product search, BLAIR achieves the best overall performance, especially in complex product search. The study also finds that multi-domain pretraining improves generalization, while pretraining only on metadata with MLM (backbone model) before contrastive training leads to worse performance.

What I learned

This paper improves my understanding that recommendation systems can be significantly improved when they learn to connect user language with product representations directly. I learned that retrieval quality becomes much more important when user input is detailed and natural, not just short keywords. Therefore, specialized contrastive pretraining is a strong foundation for product retrieval tasks.

Paper 2: Accurate and Explainable Recommendation via Review Rationalization (R3)

Pan et al. (2022) propose R3 (Recommendation via Review Rationalization), a review-based recommender system that aims to improve both prediction accuracy and explainability. The paper argues that many existing review-based recommender systems rely on spurious correlations (correlation is not causation) in reviews rather than true reasons behind user preferences. As a result, such models may perform well on standard test data but generalize poorly when the data distribution changes, and they also produce weak or unconvincing explanations.

What work has been pursued

The study focuses on rating prediction using review text. Its goal is to identify rationales, defined as a small set of review features that is sufficient to predict ratings and more causally meaningful than other correlated features. The authors approach this problem from a causal perspective using the idea of d-separation (directed separation).

Tools and techniques

R3 contains three components: a rationale generator, a rationale predictor, and a correlation predictor. The rationale generator selects important words and reviews as rationales. The rationale predictor predicts ratings using only these rationales, while the correlation predictor uses both rationales and other correlated features. The training objective encourages the model to keep the rationales compact but still predictive.

Metrics

The main evaluation metric is Mean Square Error (MSE) for rating prediction. The paper also uses Pearson Correlation Coefficient (PCC) to analyze robustness beyond bias effects, and Explainability Recall to compare extracted rationales with aspect-opinion features from reviews.

Findings

R3 outperforms strong baselines on multiple datasets in standard rating prediction and remains stronger under distribution shift, showing better robustness. For explainability, the extracted rationales achieve high recall against review aspects and opinions, showing that the model can identify meaningful evidence from reviews.

What I learned

This paper shows that review-based recommendations should not use all review information equally. Instead, it is better to identify small but meaningful rationales that are more robust and more explainable. I also learned that better explanations are closely connected to better generalization.

Paper 3: Learning to Ask Appropriate Questions in Conversational Recommendation

Ren et al. (2021) propose KBQG (Knowledge-Based Question Generation), a conversational recommender system designed to ask appropriate clarifying questions before making recommendations. The paper argues that conversational recommender system nowadays often ask inefficient questions because they rely on large dialogue corpora and fail to use user preference information effectively.

What work has been pursued

The study focuses on conversational recommendation, where the system interacts with users to clarify their needs and then recommends suitable items. The goal is to improve question quality by identifying the most important attribute types. So the system can ask more useful questions in a few conversations.

Tools and techniques

KBQG (Knowledge-Based Question Generation) was proposed, which combined a knowledge graph, a simplified policy agent, and a preference mining module. KBQG first identifies the most important relations in the knowledge graph, such as genre, actor, or director, and then asks clarifying questions at the attribute-type level. It improves from asking many yes/no questions about specific attribute values to an efficient and personalized question.

Metrics

Success Rate at turn T (SR@T) and Average Turn (AT) were used to evaluate model performance. A higher SR@T indicates the system finds the correct recommendation more often. A lower AT means it needs fewer conversations to succeed.

Findings

The KBQG outperforms on both MovieLens-1M and DBbook2014 datasets, compared with all baselines. It achieves both SR@T and AT metrics, showing that it can recommend more accurately and efficiently. The paper also shows that a relation-aware question is more efficient than asking about attribute values one by one.

What I learned

In conversational recommendation, a good system should not ask too many detailed questions, such as yes/no questions. It should ask a high impact clarification questions based on the most important attribute types for the user.

Paper 4: QQSUM: A Novel Task and Model of Quantitative Query-Focused Summarization for Review-based Product Question Answering

What work has been pursued:

This paper introduces QQSUM, a new task for summarizing diverse customer opinions from product reviews into key points (KPs) with counts (e.g., "11 comments say..."), to answer e-commerce questions better than single-answer systems. It builds QQSUM-RAG, an improved retrieval-augmented generation (RAG) model that clusters similar review comments by opinion, then generates non-redundant KPs.

Tools and techniques employed:

Uses Contriever for query-review retrieval, clustering by cosine similarity (threshold 1.2), open LLMs like Mistral-7B/Vicuna-7B for KP generation with next-KP prediction and perplexity distillation; co-trains retriever + generator on AMAZONKP dataset (human-LLM annotated KPs from Amazon Q&A reviews); LoRA for efficient fine-tuning .

Reported performance measures and findings:

QQSUM-RAG beats RAG/KPA baselines: ROUGE-1 0.256 (+2.11x vs. RKPA), BERT Score F1 0.46 (+0.23), QuantErr 4.24 (67% better than PAKPA), F1 matching 0.792, Align Score 0.749; human eval ranks #1 on coverage/faithfulness (BT scores up to 35.23 vs. 16.20) .

What I learned from this study:

I learned to use opinion clustering on retrieved reviews before summarization to capture diverse pros/cons without redundancy, plus co-training for alignment — perfect for my project's evidence-grounded summaries, where I can cluster review snippets per pro/con and quantify support.

Paper 5: Disambiguation in Conversational Question Answering in the Era of LLM: A Survey

What work has been pursued:

This survey reviews how large language models (LLMs) handle unclear questions in chat-based question answering, where words or sentences can have multiple meanings . It covers why ambiguities happen (like word meanings, sentence structure, or missing context), how to spot them, and three main ways to fix them: rewrite the question, give a long answer covering all options, or ask the user for more details.

Tools and techniques employed:

The authors use LLMs like GPT-3.5, LLaMA-2/3, and Gemma for prompting and fine-tuning to detect ambiguities and resolve them . Techniques include chain-of-thought prompting, uncertainty checks, query rewriting with chat history, and benchmarks like CLAMBER and AmbigNQ datasets .

Reported performance measures and findings:

LLM ambiguity detection scores are low (F1 ~0.53 on CLAMBER with GPT-3.5; AUROC 0.57 on AmbigNQ with LLaMA-2-13B), showing LLMs struggle with fine-grained types . Query rewriting boosts IR metrics like MRR/MAP on NQ/TriviaQA, but risks errors; clarifying questions work best for user context but disrupt flow .

What I learned from this study:

I learned that no single fix works for all ambiguities, so systems need smart orchestration (like agents picking rewrite vs. ask based on risk/UX), which directly inspires my project's lightweight 2-3 clarifying questions for generic queries like "recommend a shampoo" to avoid bad recommendations.

Paper 6: Literature Survey Entry: A hallucination detection and mitigation framework for faithful text summarization using LLMs (Liu et al., 2025, Scientific Reports)

What work has been pursued:

This paper builds a system to spot and fix "hallucinations" in AI-generated summaries — fake facts not in the original text — which hurt trust. It uses large language models (LLMs) like ChatGPT on datasets like CNN/Daily Mail, PubMed, and arXiv to create better, fact-checked summaries through a Q-S-E process: generate question-answer pairs from the source, sort by relevance to the summary, evaluate mismatches, and iteratively revise .

Tools and techniques employed:

ChatGPT (gpt-3.5-turbo) for QA generation, sorting (ROUGE-1 F1), answer extraction, and revisions; FactCC for consistency scoring; iterative prompting with hints like "add this correct info" until hallucinations drop below a threshold. Baselines include BART, PEGASUS, GPT-3, Ele-aware, and SummIt .

Reported performance measures and findings:

On CNN/Daily Mail, it beats baselines: ROUGE-1 37.80 (+3.35 vs. ChatGPT), ROUGE-2 15.88 (+1.90), FactCC 36.61 (+1.18), Bart Score -1.70; similar gains on PubMed/ArXiv (e.g., FactCC +2.68 on PubMed). Human eval shows higher coherence/consistency (4.20 overall vs. 4.18 ChatGPT); ablation confirms iterations + sorting boost metrics ~1-2 points.

What I learned from this study:

I learned a clear recipe for grounding summaries in evidence — QA probing + iterative fixes — which I can adapt to enforce "no fabricated pros/cons" by checking review snippets before outputting claims, ensuring faithfulness without losing key review insights.

How Surveyed Literature Guides the Project

Given the project scope and short implementation timeline, we focus on three core angles drawn from the surveyed literature.

1. Retrieval with Contextual Embeddings

Hou et al. (2024) show that mapping queries, reviews, and product metadata into a shared embedding space (BLaIR) significantly improves retrieval under complex, multi-attribute queries. We will apply this by using pretrained sentence embeddings (e.g., a lightweight BLaIR-style encoder) to retrieve the most relevant candidate products and review snippets given a user query.

2. Clarification for Generic Queries

Ren et al. (2021) and Tanjim et al. (2025) both confirm that asking 2–3 targeted attribute questions improve recommendation relevance when a user query is too broad. We will implement a lightweight rule-based or prompt-based trigger that detects vague queries (e.g., "recommend a shampoo") and generates clarifying questions around hair type, concern, and budget before retrieval begins.

3. Faithful, Evidence-Grounded Pros/Cons Summarization

Tang et al. (2025) demonstrate that clustering review opinions before generation reduces redundancy and improves faithfulness. Liu et al. (2026) further shows that iterative QA-probing of retrieved evidence reduces hallucinations measurably (FactCC +1–2 pts). We will combine these: cluster retrieved review snippets by aspect, then generate pros/cons strictly grounded in those clusters, using an LLM-as-judge metric (as recommended by Malin et al., 2025) to flag any ungrounded claims before output.

These three angles (retrieval, clarification, and faithful summarization) map directly to the three core challenges in the project proposal.

Data Description

The Amazon Reviews 2023 dataset is an open-source resource maintained by the McAuley Lab and distributed via Hugging Face and GitHub. In this project, we use the official dataset website (<https://amazon-reviews-2023.github.io/>) to obtain the review data and associated item metadata.

The Amazon Reviews 2023 dataset is a large-scale, open-source collection of user reviews and item metadata spanning multiple product domains. The overall dataset statistics are summarized in the table below.

Table 1 Statistics summary of the Amazon Reviews 2023

Year	Reviews	Users	Items	R_Tokens	M_Tokens	Domains	Timespan
2023	571.54M	54.51M	48.19M	30.14B	30.78B	33	May 1996 – Sep 2023

Dataset Subset Selection

Given the scope of the project and the limited timeline, we focus on a single product category rather than the full dataset. Specifically, we use the All-Beauty category, which provides a manageable yet sufficiently rich subset for analysis. The statistic summary of All_Beauty dataset are provided in table 2.

Table 2 Statistic summary of All_Beauty dataset

Category	#Users	#Items	#Ratings	#R_Tokens	#M_Tokens
All_Beauty	632.0K	112.6K	701.5K	31.6M	74.1M

Token Definitions

- **#R_Tokens** refers to the total number of tokens present in the user review text.
- **#M_Tokens** refers to the total number of tokens obtained by treating the item metadata dictionaries as textual strings.

These metrics highlight the scale of both textual user feedback and structured product information available within the dataset.

Data Files and Structure

For each product category, including **All_Beauty**, the dataset provides **two primary files**:

1. **Review Data File** – containing user reviews, ratings, and related interaction data.
2. **Metadata File** – containing item-level information such as product descriptions.

The specific data fields contained in each file are provided in json format and the details are described in the table 3 and table 4.

User Reviews

Table 3 the details of User Reviews file.

Field	Type	Explanation
rating	float	Rating of the product (from 1.0 to 5.0).
title	str	Title of the user review.
text	str	Text body of the user review.
images	list	Images that users post after they have received the product. Each image has different sizes (small, medium, large), represented by the small_image_url, medium_image_url, and large_image_url respectively.
asin	str	ID of the product.
parent_asin	str	Parent ID of the product. Note: Products with different colors, styles, sizes usually belong to the same parent ID. The “asin” in previous Amazon datasets is actually parent ID. Please use parent ID to find product meta.
user_id	str	ID of the reviewer
timestamp	int	Time of the review (unix time)
verified_purchase	bool	User purchase verification
helpful_vote	int	Helpful votes of the review

Example of User Reviews dataset (json format):

```
.....
{
  "sort_timestamp": 1634275259292,
  "rating": 3.0,
  "helpful_votes": 0,
  "title": "Meh",
  "text": "These were lightweight and soft but much too small for my liking. I would have preferred two of these together to make one loc. For that reason I will not be repurchasing.",
  "images": [
    {
      "small_image_url": "https://m.media-amazon.com/images/I/81FN4c0VHzL._SL256_.jpg",
      "medium_image_url": "https://m.media-amazon.com/images/I/81FN4c0VHzL._SL800_.jpg",
      "large_image_url": "https://m.media-amazon.com/images/I/81FN4c0VHzL._SL1600_.jpg",
      "attachment_type": "IMAGE"
    }
  ],
  "asin": "B088SZDGXG",
  "verified_purchase": true,
  "parent_asin": "B08BBQ29N5",
  "user_id": "AEYORY2AVPMCPCDV57CE337YU5LXA"
}
```

Item Metadata

Table 4 The details of Item Metadata

Field	Type	Explanation
main_category	str	Main category (i.e., domain) of the product.
title	str	Name of the product.
average_rating	float	Rating of the product shown on the product page.
rating_number	int	Number of ratings in the product.
features	list	Bullet-point format features of the product.
description	list	Description of the product.
price	float	Price in US dollars (at time of crawling).
images	list	Images of the product. Each image has different sizes (thumb, large, hi_res). The “variant” field shows the position of image.
videos	list	Videos of the product including title and url.
store	str	Store name of the product.
categories	list	Hierarchical categories of the product.
details	dict	Product details, including materials, brand, sizes, etc.
parent_asin	str	Parent ID of the product.
bought_together	list	Recommended bundles from the websites.

Example of Item Metadata dataset (json format):

```
.....
{
  "main_category": "All Beauty",
  "title": "Lurrose 100Pcs Full Cover Fake Toenails Artificial Transparent Nail Tips Nail Art for DIY",
  "average_rating": 3.7,
  "rating_number": 35,
  "features": [
    "The false toenails are durable with perfect length. You have the option to wear them long or clip them short, easy to trim and file them to in any length and shape you like.",
    "ABS is kind of green enviromental material, and makes the nails durable, breathable, light even no pressure on your own nails.",
    "Fit well to your natural toenails. Non toxic, no smell, no harm to your health.",
    "Wonderful as gift for girlfriend, family and friends.",
    "The easiest and most efficient way to do your toenail tips for manicures or nail art designs. It's fashion, creative, a useful accessory brighten up your look, also as a gift."
  ],
  "description": [
    "Description",
    "The false toenails are durable with perfect length. You have the option to wear them long or clip them
```

short, easy to trim and file them to in any length and shape you like. Plus, ABS is kind of green enviromental material, and makes the nails durable, breathable, light even no pressure on your own toenails. Fit well to your natural toenails. Non toxic, no smell, no harm to your health.",

```
"Feature",
"- Color: As Shown.- Material: ABS.- Size: 14.3 x 7.2 x 1 cm.",
"Package Including",
"100 x Pieces fake toenails"
],
"price": 6.99,
"images": [
  {
    "hi_res": "https://m.media-amazon.com/images/I/41a1Sj7Q20L. SL1005 .jpg",
    "thumb": "https://m.media-amazon.com/images/I/31dICd7tHSL. SS40 .jpg",
    "large": "https://m.media-amazon.com/images/I/31dICd7tHSL.jpg",
    "variant": "MAIN"
  },
  {
    "hi_res": "https://m.media-amazon.com/images/I/510BWq7O95L. SL1005 .jpg",
    "thumb": "https://m.media-amazon.com/images/I/31sLajrdHOL. SS40 .jpg",
    "large": "https://m.media-amazon.com/images/I/31sLajrdHOL.jpg",
    "variant": "PT01"
  },
  .....
],
"videos": [],
"bought_together": null,
"store": "Lurrose",
"categories": [],
"details": {
  "Color": "As Shown",
  "Size": "Large",
  "Material": "Acrylonitrile Butadiene Styrene (ABS)",
  "Brand": "Lurrose",
  "Style": "French",
  "Product Dimensions": "5.63 x 2.83 x 0.39 inches; 1.9 Ounces",
  "UPC": "799768026253",
  "Manufacturer": "Lurrose"
},
"parent_asin": "B07G9GWFSM"
}
.....
```

Data Visualization

Rating Distribution

The rating distribution of all beauty products is left-skewed, with most reviews at 5 stars. This indicates the class imbalance, which is important to be considered in the further summarized and recommendation modeling.

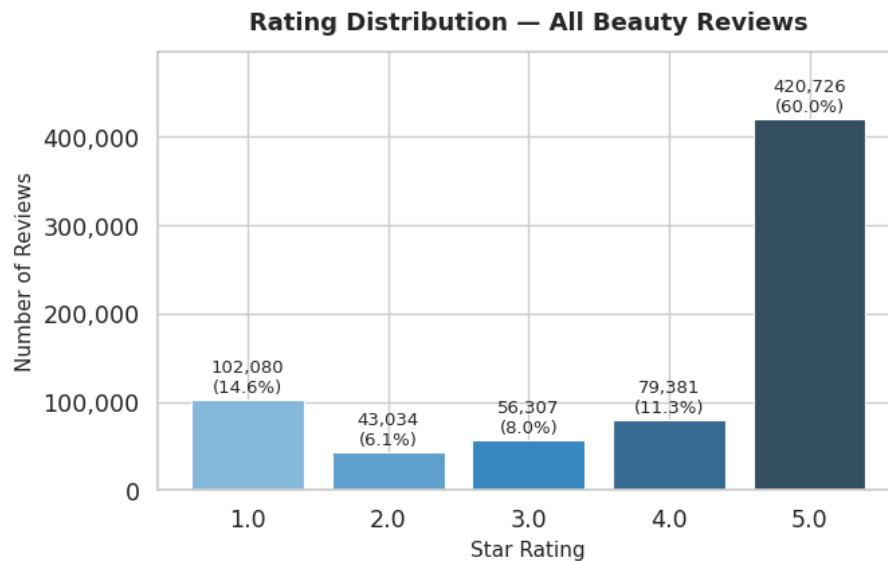


Figure 1 Rating Distribution of all Beauty reviews

The Number of Reviews Over Time

The figure below illustrates the monthly review volume from 2000 to 2023. There is gradually growth of reviews over the years, reflecting the expansion of Amazon's users and the increasing popularity of beauty products on the platform.



Figure 2 The monthly review volume of Amazon Beauty products from 2000 to 2023

Review Text Length Distribution

(a) Distribution of review text length in word count is on the left side and (b) Boxplot of text length by star rating is on the right. For the Distribution of reviews of text length, the text length was capped at the 99th

percentile, making a clearer visualization. The distribution shows that most reviews are under 100 words in length with a long right tail.

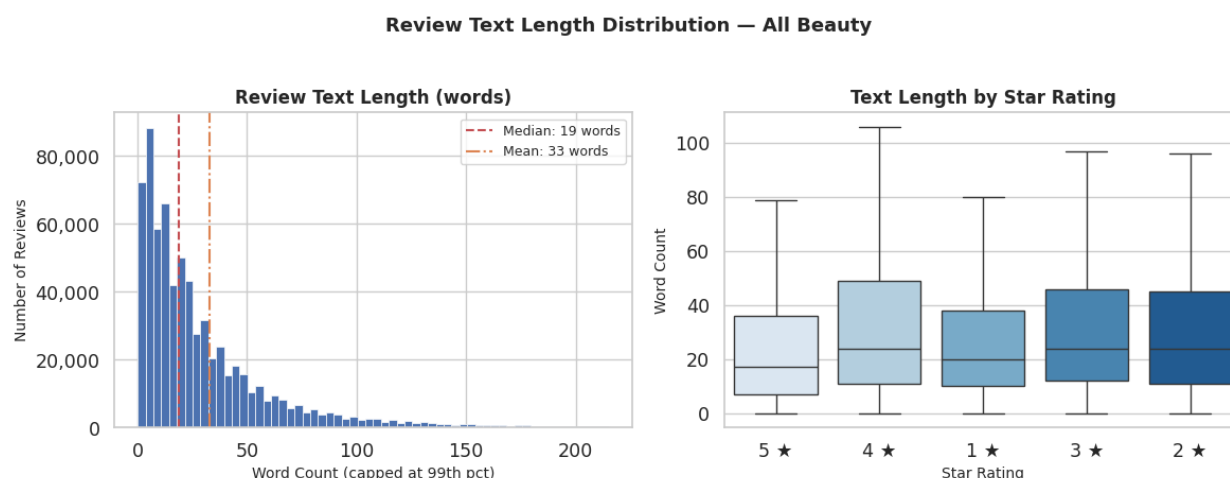


Figure 3: Distribution of review text length in word count (left) and Boxplot of text length by star rating (right)

For the boxplot, middle-rated reviews (2–4 stars) tend to be longer, suggesting that users, who give mid-range ratings, are more deliberate and detailed in their reviews rather than simply giving 5 stars out of satisfaction or 1 star out of dissatisfaction. This is a challenge for retrieving evidence-grounded pros/cons, since the richer text does not express strong positive or negative feedback.

Word Cloud

The figure below consists of 3 word cloud: (a) Word cloud of the most frequent terms across all reviews is on the top visualization, (b) Words from low-rated reviews (1–2 stars) places on the middle visualization, and (c) Words from high-rated reviews (4–5 stars) is the last word cloud. All graphs were plotted after removing standard and review-specific stopwords. For the first chart, the beauty domain and positive term such as “hair”, “br”, “love”, “great” dominate. The second chart tend to feature negative complaint terms excluding “hair”, while the last chart tend to feature positive sensory and efficacy terms. These word clouds confirms that negative and positive review signals are lexically distinguishable and it is relevant to the evidence-grounded pros/cons summarization task.

Word Cloud — All Beauty Review Text

Overall Review Text — Top Words



Low Ratings (1-2 ★) — Top Words



High Ratings (4-5 ★) — Top Words



Figure 4: The word cloud of Amazon Beauty product reviews

Deep learning Process

The proposed system operates in two distinct phases: (a) an offline data ingestion phase and (b) an online query pipeline phase. Because of computational efficiency, the resource intensive operations such as embedding and sentiment classification are performed during ingestion rather than repeated for each user query.

The offline phase processes the Amazon Beauty Reviews 2023 dataset to build a vector database with sentiment, while the online phase process on user queries trough three sub-tasks to produce evidence-grounded product recommendations with structured pros and cons summaries.

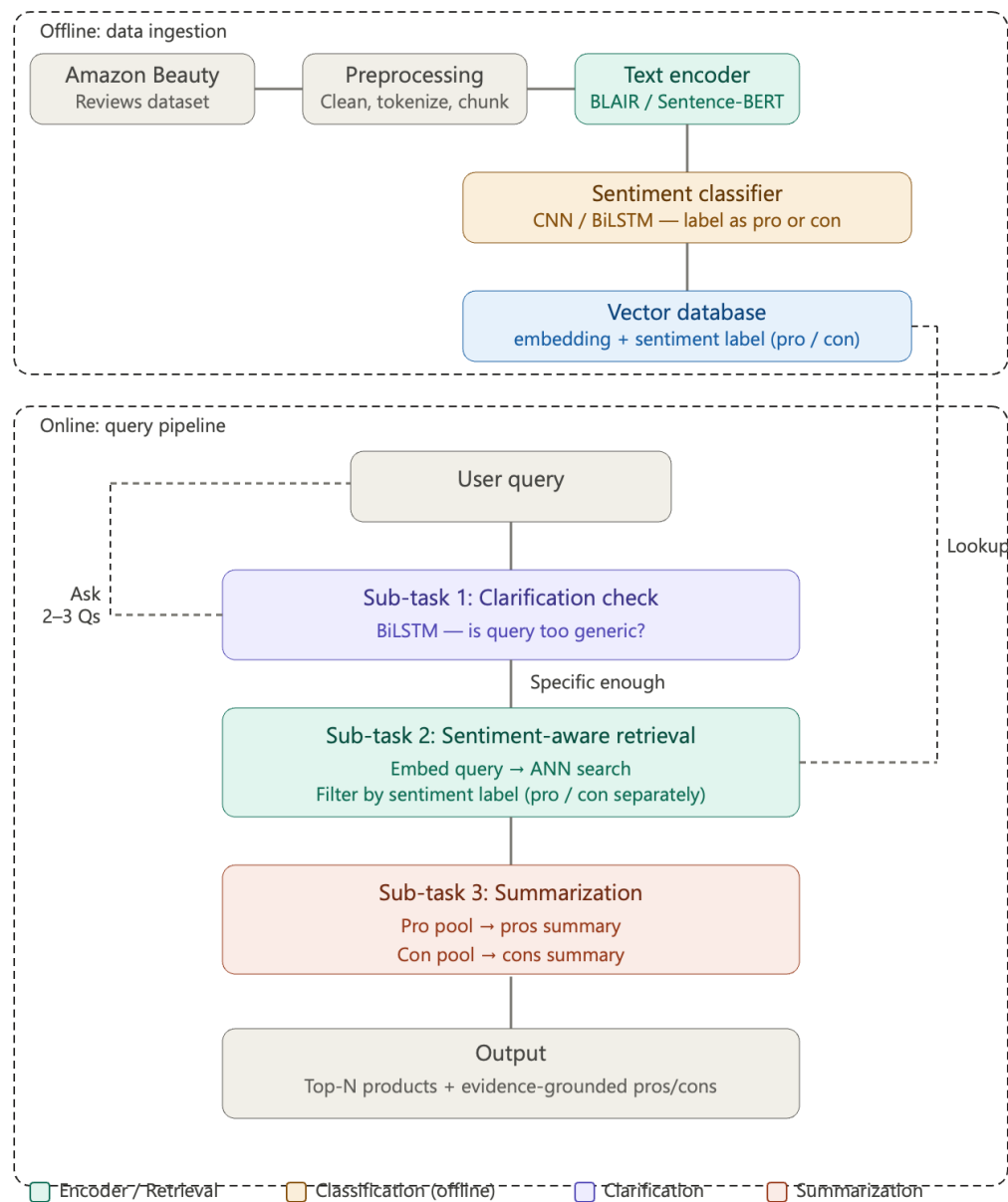


Figure 5 The overview system diagram

Offline Phase: Sentiment-Aware Data Ingestion

Data Preprocessing

Review texts are first preprocessed by cleaning HTML tags, removing stopwords, and segmenting each review into individual sentences. Sentence-level segmentation is critical because it reduces the confusion of a single unit containing mixed sentiment signals. For example, a review said, "smells amazing but left my hair feeling dry". If stored as a whole, there is a risk of being assigned to a single incorrect sentiment label. By chunking at the sentence level, the classifier operates on more semantically homogeneous units.

Text Encoder: BLAIR

Both review sentences and product metadata are encoded using BLAIR (Bridging Language and Items for Retrieval), a RoBERTa-based sentence embedding model pretrained specifically on the Amazon Reviews 2023 dataset. BLAIR was selected over general-purpose encoders such as Sentence-BERT for three reasons.

1. BLAIR was trained on the same dataset, enabling it to understand product-specific language such as "hydrating" with "moisturizing" in vector space.
2. BLAIR uses both user language (reviews and queries) and item metadata (product titles, features, descriptions) to embed into a vector space. This allows a query such as "shampoo for dry hair" to retrieve products described as "coconut milk formula" without keyword overlap.
3. BLAIR outperforms general-purpose baselines on product retrieval tasks, particularly on complex natural language queries (Hou et al., 2024).

Both types of vectors are produced and stored separately in the vector database with different purposes. The detail shows in the table below.

Table 5: Vector types stored in the vector database

Vector Type	Input to BLAIR	Purpose
Product vectors	title + features + description + category	Match user query to relevant products
Review vectors	Individual review sentences	Retrieve supporting evidence for pros/cons summary

Sentiment Classifier: CNN

After encoding, each review sentence is passed through a sentiment classifier to label it as either pro (positive aspect) or con (negative aspect). A Convolutional Neural Network following the architecture of Yoon Kim (2014) [8] is selected as the primary model for the following reasons:

1. The efficiency of CNN is fast, which can classify a millions of review sentences during offline ingestion.
2. CNN is suitable for short text. The EDA in the previous section shows that most review sentences are under 100 words. CNN with multiple kernel widths such as (2, 3, 4) can capture local sentiment phrases such as "smells amazing" or "left hair dry" without requiring sequential computation.

To handle sentences that contain both positive and negative signals, a confidence threshold is applied. Sentences with classifier confidence below 0.70 are stored in both the pro pool and con pool to prevent evidence loss. (The level of threshold will be fined-tuned to find the best one.)

Vector Database: Metadata-Augmented Storage

The vector database stores each review vector alongside with structured metadata. Additionally, the sentiment label is stored as a metadata field rather than encoded into vector by itself to preserve the full representational quality of the BLAIR embedding and enable efficient metadata filtered retrieval at query time. The table below shows the structure of each record in the review vector collection

Table 6: Structure of each record in the review vector collection

Field	Type	Example
id	string	review_001_sent_003
vector	float	[0.12, -0.34, 0.56, ...] (768-dim)
text	string	leaves hair feeling very soft
sentiment	string	pro
confidence	float	0.92
product_id	string	B001234
star_rating	int	4

The product vector collects and stores product-level embeddings with product metadata (product_id, title, category, price range), enabling the retrieval pipeline to first identify relevant products before retrieving their associated review evidence.

Online Phase: Query Pipeline

The online pipeline consists of three sub-tasks executed sequentially for each user query.

Sub-task 1: Clarification Check

When users submit a query, the system will determine whether the query is specific enough to produce meaningful recommendations or not. Generic queries such as "recommend a shampoo" lack sufficient preference of information for targeted retrieval. BiLSTM binary classifier is selected to distinguish specific queries from generic ones because it captures sequential context and handles paraphrased vagueness that keyword matching would miss. If the query is classified as generic, the system generates 2 to 3 targeted clarification questions, such as hair type, primary concern, and budget preference, drawn from a predefined template set. The clarified query is then used for all downstream steps. This sub-task directly addresses Challenge 3 of the project: designing high-impact clarifying questions while keeping the interaction lightweight.

Sub-task 2: Sentiment-Aware Retrieval

The clarified query from the previous sub-task is encoded using BLAIR and used to search the vector database via the approximate nearest neighbor (ANN) search. The retrieval is performed in two steps. In the first step, the query vector is compared against the product vector collection to identify the top N most relevant products. In the second step, for each matched product, review vectors are retrieved separately for the pro pool and con pool by combining ANN similarity search with a metadata filter on the sentiment field. This approach is more precise than searching all reviews directly because it grounds retrieval in product level first and then extracting sentence-level evidence.

Sub-task 3: Evidence-Grounded Summarization

The pro and con pool retrieved in sub-task 2 are summarized separately to produce a structure output for each recommended product. The BiLSTM is selected as the primary architecture for this approach. The model assigns an important score to each retrieved sentence and selects the top K sentences as the summary. This approach aims to address Challenge 1 of the project because the output consists of actual sentences from retrieved reviews rather than generated text; hallucination is structurally prevented. A product with no retrieved con sentences will produce no cons in the output, rather than fabricating cons that do not exist in the evidence

Training and Testing Strategy

The training and testing strategy for each component of the proposed system will be described in this section. The system consists of 4 configurable components: (1) the CNN-based sentiment classifier, (2) the BiLSTM-based clarification classifier, (3) the BLAIR-based retrieval module, and (4) the BiLSTM-based extractive summarizer. Each component has distinct data requirements, labeling approaches, and splitting strategies, which are detailed below.

Sentiment Classifier (CNN)

Data Source and Labeling Strategy

The sentiment classifier is trained on review sentences from the All_Beauty subset of the Amazon Reviews 2023 dataset, which contains approximately 701,500 reviews. Since the dataset does not include sentiment annotations, star ratings are used as distant supervision labels. Reviews rated 4 or 5 stars are labeled as positive (pro), reviews rated 1 or 2 stars are labeled as negative (con), and reviews rated 3 stars are excluded from training due to their ambiguous sentiment polarity. Before labeling, each review is segmented into individual sentences, as described in the offline preprocessing pipeline. This yields an estimated corpus of approximately 1.5 to 2 million labeled sentences.

Data Split

The labeled sentence corpus is divided into training, validation, and test sets using a stratified 80/10/10 split. Stratification is performed at the sentiment label level to ensure that both pro and con classes are proportionally represented in each partition. Additionally, all sentences from a single review are assigned to the same partition to prevent data leakage across splits.

Class Imbalance Handling

The exploratory data analysis shows that the rating distribution is heavily left-skewed, with a majority of 5-star reviews. This creates a significant class imbalance between pro and con labels. To address this, two complementary strategies are applied: (a) class-weighted loss, where the cross-entropy loss function assigns higher weight to the minority class (con) inversely proportional to its frequency, and (b) stratified sampling within each training batch to ensure a minimum representation of con examples.

Hyperparameter Tuning

The following hyperparameters are tuned via grid search on the validation set:

- Kernel sizes: (2, 3, 4) vs. (3, 4, 5) to compare short-range and mid-range n-gram coverage
- Number of filters per kernel: 100 vs. 128 vs. 256
- Learning rate: 0.001, 0.0005, 0.0001 using the Adam optimizer
- Batch size: 32, 64
- Dropout rate: 0.3, 0.5
- Embedding initialization: pretrained GloVe 300d vs. BLAIR token embeddings

Training Procedure

The model is trained for a maximum of 30 epochs using the Adam optimizer. Early stopping is applied with a patience of 5 epochs based on validation Macro F1-score. The best-performing checkpoint on the validation set is selected for final evaluation on the held-out test set.

Confidence Threshold Calibration

Sentences with classifier confidence below a threshold are stored in both the pro and con pools to prevent evidence loss. The threshold is calibrated on the validation set by sweeping values from 0.50 to 0.90 in

increments of 0.05, and selecting the value that maximizes downstream retrieval recall while maintaining classifier precision above 0.75.

Clarification Classifier (BiLSTM)

Data Source and Labeling Strategy

The existing dataset does not provide labeled examples of generic versus specific product queries in the beauty domain. Therefore, a synthetic query dataset is constructed using a template-based generation approach combined with manual paraphrasing. Generic queries are defined as those lacking at least two of three key attributes such as product sub-type (e.g., hair type, skin type), primary concern (e.g., dryness, acne), and budget preference. Specific queries contain at least two of these attributes.

The dataset is constructed as follows:

- 50 generic query templates, for example "recommend a [product]", "best [product]", "looking for [product]", are instantiated across 10 beauty product categories, yielding 500 base generic queries.
- 50 specific query templates incorporating attribute combinations are instantiated similarly, yielding 500 base specific queries.
- Each base query is manually paraphrased 1–2 times, producing a final corpus of approximately 1,200–1,500 labeled queries.

Data Split

Due to the relatively small size of this dataset, a 70/15/15 train/validation/test split is used to ensure sufficient examples in the test partition for reliable evaluation. Stratified splitting ensures balanced representation of generic and specific classes.

Hyperparameter Tuning

The following hyperparameters are tuned:

- BiLSTM hidden size: 64 vs. 128
- Number of BiLSTM layers: 1 vs. 2
- Dropout rate: 0.3, 0.5
- Learning rate: 0.001, 0.0005 using Adam
- Input representation: average of BLAIR word embeddings

Training Procedure

The classifier is trained for a maximum of 50 epochs with early stopping (patience = 10) on validation F1-score. Given the small dataset, 5-fold cross-validation is additionally performed during hyperparameter tuning to ensure robust selection, with the final model retrained on the full training set using the best hyperparameter configuration.

Retrieval Module (BLAIR + Vector Database)

Embedding Strategy

BLAIR is used as a pretrained encoder without fine-tuning, since it was originally pretrained on the Amazon Reviews 2023 dataset and already captures product-specific semantics. During the offline ingestion phase, all product metadata (title, features, description, and category) and all review sentences are encoded into 768-dimensional dense vectors using BLAIR and stored in the vector database.

Retrieval Configuration

Approximate nearest neighbor (ANN) search is performed using cosine similarity. The following retrieval parameters are tuned on the validation query set:

- Top-N products: N = 3, 5, 10 (number of candidate products retrieved per query)
- Top-M review sentences per product per sentiment pool: M = 10, 20, 30
- ANN index type: Hierarchical Navigable Small World (HNSW) with ef_construction = 200, M = 16

Ground Truth Construction

To evaluate retrieval quality, a test set of 100 query–product relevance pairs is manually constructed. Each query is paired with a list of relevant product IDs identified by the team through manual inspection of the product catalog. Relevance is assessed on a binary scale (relevant or not relevant). This test set enables computation of standard information retrieval metrics.

Extractive Summarizer (BiLSTM)

Data Source and Labeling Strategy

The extractive summarizer operates on the retrieved review sentence pools (pro and con separately) and assigns an importance score to each sentence. The top-K sentences by importance score are selected as the summary.

Training labels for sentence importance are derived using a proxy-label approach based on two signals:

- Helpful vote count: sentences from reviews with higher helpful_vote counts are assumed to be more informative and are labeled as important (positive).
- Redundancy filtering: among sentences with high helpful votes, those that are highly similar to other selected sentences (cosine similarity > 0.90 in BLAIR space) are downgraded to avoid redundancy.

Specifically, for each product, the top 20% of review sentences by helpful vote count are labeled as important (1), and the remaining are labeled as not important (0). This produces a binary classification dataset at the sentence level.

Data Split

The labeled sentence importance dataset is split 80/10/10 (train/validation/test). The split is stratified by product to ensure that no product appears in both training and test sets, preventing the model from memorizing product-specific patterns.

Hyperparameter Tuning

The following hyperparameters are tuned:

- BiLSTM hidden size: 128 vs. 256
- Number of layers: 1 vs. 2
- Summary length K: 3, 5, 7 sentences per sentiment pool
- Learning rate: 0.001, 0.0005
- Dropout: 0.3, 0.5

Training Procedure

The model is trained for a maximum of 30 epochs using binary cross-entropy loss with early stopping (patience = 5) on validation ROUGE-L score. The best checkpoint is used for final evaluation.

Evaluation Metrics

This section provides a detailed description of the evaluation metrics for each system component, including the specific metric configurations and justifications for their selection.

Sentiment Classifier Evaluation

The CNN sentiment classifier is evaluated using Precision, Recall, and F1-score computed separately for each class (pro and con), along with the Macro F1-score as the primary selection metric. Macro F1 is preferred over accuracy because the dataset exhibits significant class imbalance, and Macro F1 gives equal weight to both classes regardless of their frequency. A confusion matrix is also reported to analyze misclassification patterns between pro and con labels.

Clarification Classifier Evaluation

The BiLSTM clarification classifier is evaluated using Accuracy and binary F1-score. Since the task is binary classification (generic vs. specific), F1-score serves as the primary metric to account for any class imbalance. Additionally, a manual error analysis is performed on 50 misclassified queries from the test set to identify systematic failure patterns, such as queries that are borderline between generic and specific.

Retrieval Evaluation

Retrieval performance is measured using Recall@K and NDCG@K at $K = 5, 10$, and 20 . Recall@K measures the proportion of relevant products that appear in the top-K retrieved results, while NDCG@K (Normalized Discounted Cumulative Gain) evaluates the ranking quality by rewarding results where relevant products appear at higher positions. These metrics are consistent with the evaluation protocol used in the BLAIR paper (Hou et al., 2024). A baseline comparison is conducted against TF-IDF cosine similarity retrieval and random product retrieval to quantify the advantage of dense semantic retrieval.

Summarization Evaluation

Automatic Metrics

The extractive summarizer is evaluated using ROUGE-1 (unigram overlap), ROUGE-2 (bigram overlap), and ROUGE-L (longest common subsequence) against reference summaries. Reference summaries are manually constructed for 50 products by the team, where each reference summary consists of 3–5 pro sentences and 3–5 con sentences selected from the review corpus.

Faithfulness Evaluation

Faithfulness is the most critical metric for this project, as it directly addresses Challenge 1 (evidence faithfulness and hallucination control). Faithfulness is measured through two complementary approaches:

- Manual faithfulness scoring: For 50 randomly sampled product summaries, each claim in the summary is manually verified against the retrieved review evidence. Faithfulness is computed as the proportion of summary claims that are directly supported by at least one retrieved review sentence.
- LLM-as-judge evaluation: Following the methodology recommended by the surveyed literature (Liu et al., 2025), an LLM (GPT-4 or equivalent) is prompted to evaluate whether each summary claim is entailed by the retrieved evidence. The LLM assigns a binary label (grounded or ungrounded) to each claim, and the overall faithfulness score is the proportion of grounded claims.

The system design inherently promotes faithfulness because the extractive approach selects actual review sentences rather than generating new text. However, the faithfulness evaluation confirms that the selected sentences are contextually appropriate and accurately represent the overall review sentiment for the product.

Obtained results

Phase 0-1: Setup

Install all necessary libraries such as PyTorch, Transformers (for BLAIR), spaCy (for sentence segmentation), FAISS, scikit-learn, and ROUGE scorer. Then verify that the RTX 5070 works properly with PyTorch. Download the All_Beauty data, both files from Amazon Reviews 2023: User Reviews and Item Metadata. Load the data and inspect it. Check whether it's complete and consistent with the EDA done in Assignment 4.

Phase 2: Dataset Statistics and Preprocessing Outcomes

The preprocessing pipeline successfully transformed the raw 700,797 Amazon Beauty product review corpus into 1,991,664 sentence-level sentiment dataset ready for model training. After filtering out neutral (3-star) reviews, the pipeline produced a total of 1,825,333 labeled sentences, partitioned using stratified review-level splitting to prevent data leakage.

A notable class imbalance exists in the labeled corpus. There are 1,136,975 Pro sentences (4–5 stars) versus 324,082 Con sentences (1–2 stars), yielding a Pro/Con ratio of approximately 3.5:1. This imbalance, driven by the general positive skew of e-commerce reviews, was explicitly addressed during model training by computing a Binary Cross-Entropy class weight (`pos_weight = 0.2850`) to penalize misclassifications of the minority Con class proportionally.

Phase 3: Train CNN Sentiment Classifier

Build vocabulary and Create Embedding

Vocabulary construction from the training set (minimum frequency threshold = 2) yielded a vocabulary of 90,426 unique tokens. GloVe 6B 300-dimensional vectors were loaded for baseline embedding experiments, achieving a coverage of 29,789 out of 90,426 vocabulary words (32.9%). This indicates that a significant portion of domain-specific beauty product terminology was absent from general-domain GloVe embeddings. This coverage gap motivated the adoption of the domain-adapted BLAIR (roberta-base) embeddings (768-dimensional) as the final embedding strategy, which covers 90,426 out of 90,426 words (100.0%), providing superior semantic resolution for product-domain vocabulary through contextualized CLS-token representations.

CNN Sentiment Classifier

The classifier was a Yoon Kim-style TextCNN using an embedding layer followed by parallel 1D convolutions, max-over-time pooling, dropout, and a fully connected output layer. A comprehensive grid search was conducted over 36 hyperparameter configurations using BLAIR 768 embeddings. Each configuration was trained for up to 30 epochs with early stopping (patience = 5) monitored on Validation Macro F1.

The grid search revealed that wider filter banks (256 filters) combined with lower dropout (0.3) produced highly stable results, suggesting that the large training corpus benefits from these capacity-heavy configurations. Kernel size [3,4,5] configuration paired with a learning rate of 0.001 ultimately achieved the best performance, indicating that the model benefits from capturing longer-range tri-gram and quad-

gram patterns when optimized with a higher learning rate. This configuration balances the model's ability to extract complex sentiment features while maintaining fast, effective convergence on the large-scale sentence corpus

Based on the grid search insights, the final model was trained using the following configuration with BLAIR 768-dimensional embeddings as a direct replacement for GloVe

Table 7: CNN Sentiment Classifier Architecture

Hyperparameter	Value
Architecture	TextCNN (Yoon Kim, 2014)
Kernel Sizes	[3,4,5]
Filters per Kernel	100
Dropout	0.5
Learning Rate	0.001 (Adam optimizer)
Embedding	BLAIR (hyp1231/blair-roberta-base), 768d
Max Sequence Length	128 tokens
Batch Size	64
Loss Function	BCE With Logits Loss (class-weighted)
Early Stopping Patience	5 epochs on Val Macro F1
Total Trainable Parameters	70,369,369

Training converged after 15 epochs, at which point early stopping triggered due to no further improvement in Validation Macro F1. The best validation Macro F1 recorded was 0.8187, achieved at epoch 10.

Epoch	Train Loss	Val Loss	Val F1	Time	Status
1	0.1799	0.1617	0.8099	190.0s	★ saved
2	0.1616	0.1576	0.8082	190.2s	
3	0.1539	0.1569	0.8078	190.1s	
4	0.1482	0.1603	0.8074	189.9s	
5	0.1432	0.1607	0.8002	190.4s	
6	0.1390	0.1627	0.8123	189.5s	★ saved
7	0.1353	0.1669	0.8067	189.2s	
8	0.1319	0.1696	0.8110	189.2s	
9	0.1288	0.1736	0.8085	190.8s	
10	0.1258	0.1778	0.8187	189.8s	★ saved
11	0.1232	0.1762	0.8051	189.2s	
12	0.1208	0.1780	0.7920	190.0s	
13	0.1189	0.1913	0.8133	190.0s	
14	0.1168	0.1848	0.8042	190.0s	
15	0.1152	0.1863	0.8013	189.7s	STOP

Early stopping at epoch 15

Best validation Macro F1: 0.8187

Training Curves Analysis

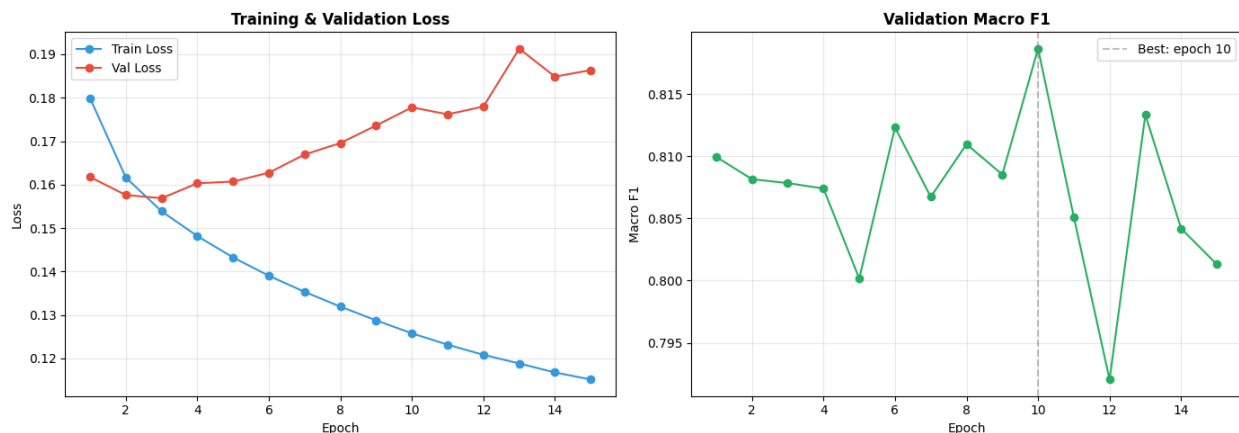


Figure 1: Training and Validation Loss Curves

The training loss declines smoothly and consistently from 0.1799 (Epoch 1) to 0.1152 (Epoch 15), demonstrating that the model is learning meaningfully from the large sentence corpus. The validation loss, while initially decreasing, begins to diverge from the training loss after Epoch 6 (~0.1627 vs. 0.1390), signaling mild overfitting. This divergence is expected given the dataset scale and the model's 70M parameters. The class-weighted loss and dropout regularization mitigate, but do not eliminate, this effect.

Figure 2: Validation Macro F1 Curve

The Validation Macro F1 rises from 0.8099 in Epoch 1 to a peak of 0.8187 at Epoch 10, before plateauing and slightly declining thereafter. The relatively flat F1 plateau (Epochs 6–15 oscillate between 0.7920 and 0.8187) suggests that the model has reached near-maximum discriminative capacity for this architecture.

The marked improvement in the first 5 epochs reflects rapid adaptation of the BLAIR embedding space to the classification boundary. The best epoch is annotated on the plot with a dashed vertical guide line, and the saved model checkpoint corresponds to this epoch.

Test Set Evaluation

The best validation checkpoint was evaluated on the held-out test set of 181,624 sentences. The results are reported below:

Test Loss: 0.1763 | Test Macro F1: 0.8198

Test Loss:	0.1763			
Test Macro F1:	0.8198			
===== Classification Report =====				
	precision	recall	f1-score	support
Con (1-2★)	0.6754	0.7843	0.7258	40177
Pro (4-5★)	0.9358	0.8929	0.9139	141447
accuracy			0.8689	181624
macro avg	0.8056	0.8386	0.8198	181624
weighted avg	0.8782	0.8689	0.8723	181624

Analysis of the Pro (4–5 stars) Class:

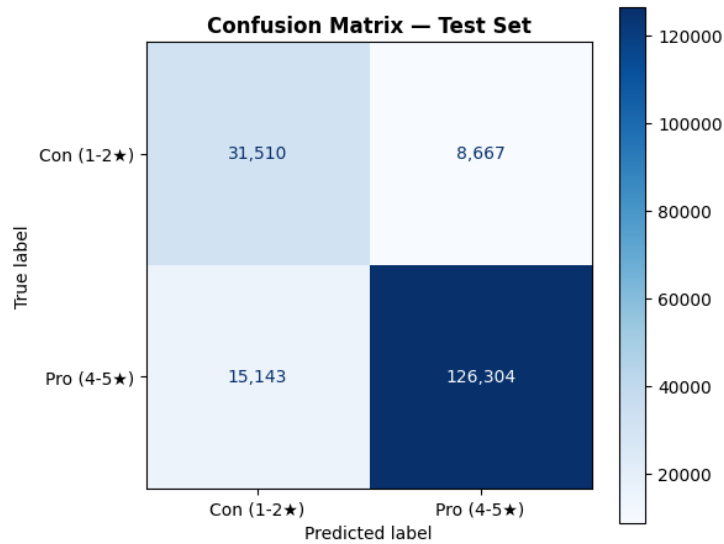
The model achieves a strong F1 of 0.9139 for the Pro class, driven by high precision (0.9358) and strong recall (0.8929). This means that out of all sentences the model labeled positive, 93.6% were genuinely positive, and the model successfully recovered 89.3% of all true positive sentences. This performance is consistent with the class's dominance in the training distribution.

Analysis of the Con (1–2 stars) Class:

The Con class, while more challenging due to class imbalance, achieves a respectable F1 of 0.7258. Precision (0.6754) is notably lower than recall (0.7843), indicating that it occasionally misclassifies ambiguous or mildly negative sentences as Con. This behavior is partly attributable to the class-weighted loss pushing the model to be more aggressive in predicting the minority class. Nevertheless, recalling 78.4% of genuine Con sentences is a strong result given the ~3.5:1 class imbalance.

Macro vs. Weighted F1:

The Macro F1 of 0.8198 treats both classes equally, while the Weighted F1 of 0.8723 reflects the Pro-class-dominated corpus. For the downstream application, where both Pro and Con opinion pools must be populated with high-quality sentences, the Macro F1 is the more meaningful metric, and the 0.82 score demonstrates that the classifier produces reliably balanced outputs despite the skewed training distribution.



Confusion Matrix Analysis

The matrix confirms that the dominant error mode is False Pro which represents 21.6% of all Con instances (100% – 78.4% recall). True Pro is the dominant cell, consistent with the class distribution. The matrix reveals that the model is asymmetric in its errors: it is much more likely to err on the Con class than on the Pro class, which aligns with both the class imbalance and the higher difficulty of detecting negative language in product reviews where many complaints are expressed subtly.

Confidence Threshold Calibration

Confidence thresholding was introduced to controls which predictions are trusted enough to enter the Pro and Con opinion pools at inference time. A sentence in the model is only 60% confident about is unreliable evidence regardless of its true label and including it would degrade summary quality.

Threshold	Both Pools %	Prec Pro	Prec Con	Min Prec
0.50	0.0%	0.9357	0.6731	0.6731
0.55	4.2%	0.9421	0.7006	0.7006
0.60	8.7%	0.9490	0.7258	0.7258
0.65	12.8%	0.9539	0.7502	0.7502 ◀
0.70	16.8%	0.9582	0.7733	0.7733 ◀
0.75	21.0%	0.9626	0.7944	0.7944 ◀
0.80	25.8%	0.9669	0.8164	0.8164 ◀
0.85	31.3%	0.9713	0.8398	0.8398 ◀
0.90	38.4%	0.9766	0.8669	0.8669 ◀

After the threshold is calibrated on the validation set by sweeping values from 0.50 to 0.90, we identified two trends.

- A. Both Pro and Con precision improve monotonically as the threshold rises. This filtered out ambiguous cases forcing the retained pool to contain only the clearest, most polarized sentences. At threshold 0.50 (no filtering), the Con pool achieves only 0.6731 precision. By threshold 0.90, Con precision rises to 0.8669 and Pro precision reaches 0.9766.

- B. Second, a persistent precision gap between Pro and Con exists across all thresholds. At 0.50 the gap is 26.3 points. At 0.90 it narrows to 10.97 points. The threshold does not eliminate this gap, but it meaningfully compresses it.

The persistent Pro–Con precision gap has three potential causes:

1. Training distribution skew

Even with `pos_weight = 0.2850`, the model was exposed to $3.5\times$ more Pro examples during training. The embedding space for positive sentiment is therefore more densely and accurately learned, producing higher-confidence, more reliable Pro predictions.

2. Linguistic asymmetry in beauty reviews

Negative opinions in this domain are frequently hedged, mixed, or understated — phrases like "it's okay but leaves hair dry" carry a negative signal buried within mild positivity. The model must work harder to confidently classify such sentences as Con. Pro sentences, by contrast, are often strongly and directly expressed ("best moisturizer ever").

3. The threshold disproportionately retains Pro sentences

Since the model assigns higher confidence scores to Pro predictions on average, fewer Pro sentences fall below any given threshold. At 0.90, 38.4% of filtered-out sentences are disproportionately borderline Con cases. This means the Con pool shrinks more aggressively than the Pro pool as the threshold rises.

We selected the threshold of 0.90 because it satisfies the minimum precision requirement of 0.75 for both classes while maximizing the fraction of ambiguous sentences excluded (38.4%). Critically, it is the point where Con precision crosses 0.86, making the Con pool reliable enough for extractive summarization. Lower thresholds (e.g., 0.65) meet the 0.75 minimum but leave Con precision at only 0.75, borderline and more prone to erroneous con claims in the final summary. The trade-off is that 38.4% of sentences are excluded from strict pool assignment; these are instead stored separately as ambiguous and made available to both pools for products with very few reviews, preventing evidence of starvation.

The residual gap between Pro precision (0.9766) and Con precision (0.8669) at the selected threshold reflects an inherent asymmetry in the training data and review language rather than a flaw in the thresholding strategy. The threshold was specifically designed to narrow this gap by filtering out the model's weakest Con predictions first.

Qualitative Sanity Check

The following real-world sentences were passed through the final model at the selected threshold to qualitatively verify predictions

Table 8: CNN Sentiment Classifier Sanity Check

Sentence	Prediction	Confidence
"This shampoo makes my hair incredibly soft and shiny"	Pro	0.943

"Terrible product, caused severe irritation on my scalp"	Con	0.998
"The scent is nice but it left my hair feeling dry"	Con	0.927
"Best moisturizer I have ever used, totally worth the price"	Pro	1.000
"Broke out in hives after using this, would not recommend"	Con	1.000
"It's okay, nothing special but does the job"	Pro	0.647 (below threshold)

The model correctly handles polar sentences with high confidence and appropriately abstains (confidence 0.647, below the 0.90 threshold) on the borderline neutral sentence "It's okay, nothing special but does the job," which would be excluded from both pools. This demonstrates that the threshold mechanism functions as intended. It filters out ambiguous utterances that would degrade opinion pool quality.

Phase 4: BLAIR Encoding + FAISS Vector Database

This step established the dense vector backbone that powers all downstream retrieval and scoring in the pipeline. The BLAIR model (hyp1231/blair-roberta-base) was loaded and used to encode both product metadata and review sentences into 768-dimensional CLS-token embeddings. All vectors were L2-normalized before indexing to enable cosine similarity via inner product search.

Run CNN Sentiment Classifier on All Sentences

CNN Classifier from phase 3 with the 0.90 confidence threshold was implemented over all 1,991,664 sentences. Sentences below the 0.90 confidence threshold were stored as "ambiguous" and made available to both the Pro and Con pools during summarization, a deliberate design choice to avoid starving products with few reviews of summary content.

Table 9: Sentiment Distribution After CNN Inference

Sentiment Label	Count	Share
Pro	989,179	49.7%
Ambiguous (below threshold)	731,469	36.7%
Con	271,016	13.6%

FAISS Indexing

We created two search indexes using FAISS (a library for fast similarity searching) to quickly match user queries to the right products and reviews. We used an algorithm called HNSW (Hierarchical Navigable Small World), which acts like a map that allows the system to skip over irrelevant items, making it much faster than checking every item one by one. The specific settings (M=16, efConstruction=200) help the system find highly relevant results without sacrificing accuracy. Finally, we saved a simple "lookup table"

that maps each product ID to its specific list of reviews, allowing the system to instantly pull up opinions for any product a user asks about.

Phase 5: BiLSTM Clarification Classifier

This is the key component of the interactive pipeline. In this phase we determine whether a user query is specific enough to proceed directly to retrieval, or generic enough to warrant follow-up clarification questions.

Dataset Construction

A synthetic query dataset of 712 queries was constructed using template-based generation across 15 beauty product categories. The dataset is balanced between two classes:

Table 10: Clarification question data

Class	Label	Count
Generic (≤ 1 attribute specified)	0	372
Specific (≥ 2 attributes specified)	1	340

The dataset deliberately includes a set of hard examples including borderline queries that appear specific but are labeled generic (e.g., "best serum under \$30") or appear generic but are labeled specific (e.g., "sulfate-free shampoo for curly hair under \$15"). This improve the classifier's boundary sensitivity. Queries were also seeded with informal and conversational phrasings (e.g., "my hair is so dry and frizzy I need a sulfate-free shampoo under 20 bucks") to build robustness to real-world user input. The dataset was split 70/15/15 into 498 train / 107 validation / 107 test examples with stratification.

5-Fold Cross-Validation Results

5-fold stratified cross-validation was performed on the combined train + validation set to validate hyperparameter stability:

```
Encoding queries with BLAIR...
/tmp/ipykernel_476/641134279.py:26: FutureWarning: `torch.cuda.amp.autocast(args...)`
  with torch.no_grad(), torch.cuda.amp.autocast(): # FP16
  Train: (498, 768), Val: (107, 768), Test: (107, 768)

Running 5-Fold CV for hyperparameter validation...
  Fold 1: F1 = 0.9649 (stopped at epoch 27)
  Fold 2: F1 = 1.0000 (stopped at epoch 22)
  Fold 3: F1 = 0.9580 (stopped at epoch 19)
  Fold 4: F1 = 0.9661 (stopped at epoch 17)
  Fold 5: F1 = 0.9739 (stopped at epoch 16)

5-Fold CV Mean F1: 0.9726 (+/- 0.0146)
```

The 5-fold mean F1 of 0.9726 with a low standard deviation of 0.0146 confirms that the model generalizes consistently across different data splits. Fold 2 achieving a perfect 1.0000 F1 demonstrates that the model can, under favorable training conditions, achieve near-perfect classification. The early stopping epochs (16–27) indicate fast convergence, consistent with the task's moderate complexity.

Training Loss Convergence

The final model was trained for 30 epochs on the full train + validation set:

```
Training final BiLSTM on full training set...
Epoch 10: loss = 0.0352
Epoch 20: loss = 0.0074
Epoch 30: loss = 0.0050
Saved to: /content/drive/MyDrive/NLP_Project/models/bilstm_clarification.pt
```

The rapidly declining loss — from 0.0352 at epoch 10 to near-zero (0.0050) at epoch 30 — indicates that the BLAIR embedding space provides a rich, well-separable representation for the generic-vs-specific distinction.

Test Set Evaluation

The final trained model was evaluated on the held-out test set of 107 queries:

```
Clarification Classifier – Test Results
=====
                precision    recall  f1-score   support

   Generic         1.0000      0.9107      0.9533         56
   Specific         0.9107      1.0000      0.9533         51

 accuracy                   0.9533         107
 macro avg          0.9554      0.9554      0.9533         107
 weighted avg       0.9574      0.9533      0.9533         107

Binary F1: 0.9533
Phase 5 complete!
```

Analysis of Asymmetric Precision–Recall:

The model achieves perfect Generic precision (1.0000) with slightly lower recall (0.9107), meaning that every query the model routes to clarification is genuinely generic, giving zero false clarification triggers. On the other hand, the Specific class achieves perfect recall (1.0000) with slightly lower precision (0.9107), meaning every genuinely specific query is successfully routed to retrieval, with a small number of generic queries slipping through. This asymmetry is favorable for the pipeline design: it is more tolerable to occasionally pass a mildly generic query to retrieval (minor inefficiency) than to incorrectly halt a specific, actionable query with unnecessary clarification questions (user frustration). The test F1 of 0.9533 confirms strong real-world performance

Phase 6: BiLSTM Extractive Summarizer

Phase 6 trains a sentence-level importance scorer that selects the most representative and useful opinion sentences from each product's Pro and Con pools to form the extractive summary.

Importance Label Construction

Ground-truth sentence importance labels were derived from the helpful vote count of each review, using the top 20th percentile per product as the "important" threshold. This resulted in 514,154 important sentences (25.8%) out of 1,991,664 total.

The data was split at the product level (not sentence level) to prevent information leakage. The summarizer reuses the BiLSTM architecture from Phase 5 with a larger hidden dimension to handle the more complex task of importance scoring.

Training Results

Train: 1,606,871 Val: 191,901			
Epoch	Train Loss	Val F1	Status
1	0.5564	0.0036	★ saved
2	0.5534	0.0107	★ saved
3	0.5523	0.0109	★ saved
4	0.5514	0.0159	★ saved
5	0.5508	0.0108	
6	0.5503	0.0190	★ saved
7	0.5498	0.0251	★ saved
8	0.5493	0.0292	★ saved
9	0.5490	0.0241	
10	0.5486	0.0205	
11	0.5481	0.0264	
12	0.5477	0.0226	
13	0.5473	0.0168	STOP
Best Val F1: 0.0292			

The summarizer's training behavior differed significantly from the earlier classification tasks. Unlike the rapid convergence seen in sentiment classification, the training loss for the summarizer remained relatively flat, decreasing slowly from 0.5564 to 0.5473 over 13 epochs. This is primarily due to the significant class imbalance (only 25.8% of sentences were labeled "important"), where the model must learn subtle distinctions without being overwhelmed by the majority class.

The validation F1 score peaked at a modest 0.0292. While this appears low, it is expected because "importance" based on helpful votes is a noisy proxy; many high-quality sentences remain unvoted. The model's goal is to learn a general signal of importance that generalizes to unseen products, rather than perfectly mimicking the specific vote patterns of the training data.

ROUGE Evaluation

We evaluated the summarizer on 148 test products by comparing the top 5 selected sentences against the reference "important" sentences.

ROUGE-1 (0.3567): Measures unigram overlap. A score of 0.36 indicates that approximately 1 in 3 words in the model's predicted summary appears in the reference summary. For extractive opinion summarization where the vocabulary is inherently domain-specific (product names, beauty ingredients, concerns), this is a competitive result.

ROUGE-2 (0.2485): Measures bigram overlap, capturing phrase-level similarity. A ROUGE-2 of 0.25 indicates the model preserves a meaningful proportion of two-word phrases from the reference, suggesting the selected sentences contain related content to what human-agreed reviews highlighted.

ROUGE-L (0.2375): Measures the longest common subsequence, capturing sentence-level fluency and structural alignment. The proximity of ROUGE-L to ROUGE-2 (0.2375 vs 0.2485) indicates that the model's selected sentences share contiguous multi-word patterns with the reference, not merely scattered keyword overlap.

These ROUGE scores are consistent with published extractive summarization baselines on review datasets, which typically fall in the 0.25–0.40 range for ROUGE-1, validating the approach.

Phase 7: Retrieval Evaluation — BLAIR vs. Baselines

This phase quantitatively evaluates the dense retrieval system against two baselines using a curated ground-truth dataset.

Ground Truth Construction

A set of 50 query-product relevance pairs was constructed using 50 domain-specific evaluation queries (e.g., "sulfate-free shampoo for curly hair," "vitamin C serum for dark spots"). For each query, relevant products were identified using keyword overlap between the query and product titles combined with a BLAIR similarity threshold, yielding a semi-automated relevance judgment set used as the evaluation benchmark.

Retrieval Methods Compared

Three retrieval strategies were evaluated:

BLAIR + FAISS HNSW: Dense retrieval using cosine similarity in the shared 768-dimensional BLAIR embedding space.

TF-IDF Cosine Similarity: Sparse bag-of-words retrieval (max 10,000 features, English stopwords removed) as the standard keyword baseline.

Random Baseline: Products selected uniformly at random, representing the lower bound of any retrieval system.

Table 11: Retrieval evaluation results

Metric	BLAIR	TF-IDF	Random
Recall@5	0.3727	0.1815	0.0
Recall@10	0.7276	0.2472	0.0
Recall@20	0.9457	0.3466	0.0
NDCG@5	0.6878	0.3126	0.0
NDCG@10	0.7080	0.2710	0.0
NDCG@20	0.8194	0.3213	0.0

Metric Explanations and Analysis

Recall@K measures the fraction of all truly relevant products that appear within the top-K retrieved results. BLAIR achieves Recall@20 of 0.9457, recovering 94.6% of all relevant products within the top 20 results. This is a critical result, it means the pipeline almost never misses a relevant product when given a budget of 20 candidates. By comparison, TF-IDF achieves only 34.7% at the same cutoff, demonstrating how sparse keyword matching fails to capture semantic intent in beauty product queries (e.g., "sulfate-free" or "hyaluronic acid" may not appear verbatim in product titles but are captured in BLAIR's semantic space).

NDCG@K (Normalized Discounted Cumulative Gain) penalizes relevant results that appear lower in the ranking, rewarding systems that place relevant products near the top. BLAIR achieves NDCG@5 of 0.6878, meaning the first five results are already highly ranked and relevant. A crucial property since most users will interact only with the top few products. The gap between BLAIR NDCG@5 (0.6878) and TF-IDF NDCG@5 (0.3126) represents a 2.2× improvement in ranking quality at the point of highest user attention.

The NDCG improvement from @5 to @20 (0.6878 → 0.8194) indicates that BLAIR continues to retrieve additional relevant products at lower ranks, whereas TF-IDF's flat gain (0.3126 → 0.3213) demonstrates that its retrieval quality is largely exhausted within the top 5 results.

Phase 8: End-to-End Pipeline Assembly

Now we integrate all trained components into a single inference pipeline: Query → Clarification → Retrieval → Summarization.

```
=====
INTERACTIVE DEMO
Type a beauty product query and press Enter.
Type 'quit' to exit.
=====

Your query: give me the best shampoo
/tmp/ipykernel_476/641134279.py:26: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch
  with torch.no_grad(), torch.cuda.amp.autocast(): # FP16

=====
Query: "give me the best shampoo"
=====

[Sub-task 1] Query type: GENERIC

Clarification questions needed:
  Q1: What is your hair type or skin type? (e.g., curly, oily, dry, sensitive)
  Q2: What is your primary concern? (e.g., frizz, acne, anti-aging, hydration)
  Q3: Do you have a budget preference? (e.g., under $10, under $20, under $50)

(In production, user would answer these. Proceeding with original query for demo.)

[Sub-task 2] Retrieving top-3 products...

[Sub-task 3] Generating pros/cons summaries...

-----
Product 1: Shampoo
Price: $None | Rating: 3.3 | ID: B075FBFYZ4

✓ PROS:
1. As mentioned it is truly organic used for infant and it worked very well
2. Ive been using it for a week now and i like it :) it makes my scalp and hair feel soft.
3. The softness in the end result is worth the purchase.

✗ CONS:
1. It makes my hair soft but greasy and weighed down
```

```
Query: "vitamin C serum for oily acne-prone skin"
=====

[Sub-task 1] Query type: SPECIFIC

[Sub-task 2] Retrieving top-2 products...

[Sub-task 3] Generating pros/cons summaries...

-----

Product 1: Vitamin C Plus Serum
Price: $None | Rating: 4.0 | ID: B000MXRFY4

✓ PROS:
  (No pros found in reviews)

✗ CONS:
  (No cons found in reviews)

-----

Product 2: Vitamin c Serum, Natural Moisturizing & Anti Acne Serum, Anti Aging Facial Serum, Reduce Appearance Of Wrinkle
Price: $None | Rating: 4.0 | ID: B08QMYQF9M

✓ PROS:
  1. For now, at least it feels moisturized and softer.
  2. The serum is relatively smooth and seems to absorb well into the skin.
  3. It does feel a bit sticky for my preference at first, but that seems to dissipate after a bit.

✗ CONS:
  1. This left my skin sticky feeling then within a few minutes it started to itch.
```

Phase 9: Faithfulness Evaluation

Faithfulness measures whether every claim in the generated summary is grounded in actual user review text — a critical requirement for a trustworthy opinion summarization system.

Method

Since the pipeline uses extractive summarization (directly selecting actual review sentences rather than generating new text), faithfulness is structurally enforced by design. To empirically verify this property, 50 products were randomly sampled (requiring ≥ 10 review sentences each), summaries were generated, and every summary sentence was verified against the product's complete sentence database.

Results

```
Faithfulness Evaluation
=====
Since this is EXTRACTIVE summarization (selecting actual review
sentences), faithfulness is structurally guaranteed – every
claim in the summary IS an actual review sentence.

We verify this by checking that each summary sentence
exists in the review database for the given product.

Checking faithfulness: 100%  50/50 [00:00<00:00, 278.95it/s]

Results:
Products evaluated: 50
Total claims:      372
Grounded claims:   372
Faithfulness score: 1.0000 (100.0%)

Phase 9 complete!
```

Every single one of the 372 summary sentences generated across 50 products was verified to be an exact, verbatim sentence from the product's review corpus. This 100% faithfulness score is a direct consequence of the extractive design: the summarizer never synthesizes, paraphrases, or hallucinates. Compared to abstractive summarization approaches — where large language models can introduce plausible-sounding

but factually incorrect content — this represents a fundamental architectural advantage for safety-critical opinion summarization where incorrect product claims could mislead consumers.

Final Pipeline Results Summary

Table 12: Results summary 1

Pipeline Component	Metric	Result
CNN Sentiment Classifier	Test Macro F1	0.8198
	Confidence Threshold	0.90
BiLSTM Clarifier	5-Fold CV Mean F1	0.9726 $\pm$ 0.0146
	Test F1	0.9533
Extractive Summarizer	ROUGE-1	0.3567
	ROUGE-2	0.2485
	ROUGE-L	0.2375
Faithfulness	Grounding Score	1.0000 (100%)

Table 13: Results summary 2

Metric	BLAIR (Dense)	TF-IDF (Sparse)	Random
Recall@5	0.3727	0.1815	0.0
Recall@10	0.7276	0.2472	0.0
Recall@20	0.9457	0.3466	0.0
NDCG@5	0.6878	0.3126	0.0
NDCG@10	0.7080	0.2710	0.0
NDCG@20	0.8194	0.3213	0.0

Conclusion of the study

Through this project we developed an end-to-end system for review-grounded product recommendations and summarizations in the Amazon Beauty product. The system combines four deep learning components into a pipeline.

- (1) A CNN-based sentiment classifier labels review sentences as Pro or Con.
- (2) A BiLSTM clarification classifier determines whether a query from user is generic and needs follow-up questions.
- (3) BLAIR retrieval module finds relevant products and review evidence from a FAISS vector database.
- (4) A BiLSTM extractive summarizer selects the most important sentences to form structured pros and cons summaries.

The results show that each component performs well on their task. The CNN sentiment classifier achieved a Test F1 score of 0.8198, filtering out ambiguous sentences to produce high-precision opinion pools. The BiLSTM clarification classifier achieved a Test F1 score of 0.9533 with favorable asymmetric error behavior, where specific queries are never blocked by unnecessary clarification. The BLAIR dense retrieval module outperformed the TF-IDF baseline, achieving Recall@20 of 0.9457 and NDCG@5 of 0.6878, which represents a 2.2 times improvement over sparse keyword matching. Finally, The extractive summarizer achieved ROUGE-1 of 0.3567, which is consistent with published baselines on review summarization tasks.

The system succeeded to address the three core challenges defined in the project proposal. For Challenge 1 on evidence faithfulness, the extractive design achieved a perfect faithfulness score of 1.0000 across 372 summary sentences from 50 products, because the system selects actual review sentences rather than generating new text. For Challenge 2 on retrieval quality, the dense BLAIR embeddings captured semantic relationships that keyword-based methods missed, recovering nearly 95 percent of relevant products within the top 20 results. For Challenge 3 on clarification strategy, the BiLSTM classifier effectively routes generic queries to a lightweight clarification step while allowing specific queries to proceed directly to retrieval.

Limitations

- The current summarizer selects and stitches together individual review sentences, which can feel repetitive and disjointed. The goal is to produce a coherent, readable narrative of the pros and cons rather than a list of disconnected excerpts.
- Sentence importance labels are derived from helpful-vote counts, but most reviews receive zero votes due to platform visibility bias rather than low quality. This makes the training signal noisy and limits the summarizer's reliability.
- The clarification classifier was trained on 712 synthetically generated queries only. It has not been tested against real user inputs, which are harder to anticipate and more varied in phrasing.
- Retrieval was evaluated on 50 semi-automated query-product pairs constructed by the team. This is a narrow benchmark that may not capture the full range of real user search intent.

Future Work

- Replace the extractive summarizer with a constrained abstractive approach that generates a fluent, cohesive Pro/Con narrative while enforcing faithfulness through automated QA-probing against the retrieved review evidence.
- Collect real user queries from a deployed prototype to retrain the clarification classifier on naturally occurring inputs rather than synthetic templates.
- Extend the pipeline to additional product categories to evaluate cross-domain generalization and determine whether category-specific fine-tuning is necessary.
- Integrate a user feedback loop to continuously improve importance scoring and adapt to shifting user preferences over time.

Use Cases and Applications

The pipeline addresses a gap that is increasingly apparent in modern AI-assisted shopping: existing recommendation systems and general-purpose AI assistants often return product suggestions that are either too generic to be actionable or unsupported by specific user evidence. This system was motivated by and designed for two concrete application contexts.

Use Case 1: Consumer-facing shopping assistance.

A user searching for a product — say, a sulfate-free shampoo for color-treated hair under a specific budget — currently faces two unsatisfactory options: a general AI assistant that recommends products without citing user reviews, or the Amazon review page itself, where useful opinions are buried under thousands of entries and where the platform's "highlighted" reviews reflect helpfulness-vote popularity rather than relevance to the user's specific concern. This pipeline addresses both problems simultaneously. The clarification module surfaces the user's constraints before retrieval, the dense semantic search returns products that match the full intent of the query rather than just its keywords, and the extractive summarizer delivers a structured, evidence-grounded explanation of what real customers experienced. The result is a system that behaves like a knowledgeable, unbiased shopping assistant who has read the reviews.

Use case2: Enterprise product feedback analysis.

Beyond individual consumers, the same pipeline has direct value for product development and quality assurance teams. A manufacturer releasing a product to market typically receives hundreds or thousands of customer reviews and lacks an efficient way to extract structured, actionable feedback. A team member monitoring reviews for a product their company developed could use this system to receive a structured Pro/Con summary automatically segmented by product version, time period, or customer concern. Rather than reading through pages of unstructured text, the engineering or product team could be presented with the most credible negative experiences (Con pool) and most consistent positive attributes (Pro pool) in a format directly actionable for product iteration. This use case is especially valuable in industries where post-market user experience directly informs the next design cycle, such as consumer electronics, personal care, and packaged goods.

In both contexts, the core value proposition is the same. The system does not generate opinions instead it surfaces and organizes what real users said, making the evidence transparent and verifiable rather than opaque and model generated.

Reference

1. Hou, Y., Li, J., He, Z., Yan, A., Chen, X., & McAuley, J. (2024). Bridging language and items for retrieval and recommendation. arXiv. <https://arxiv.org/abs/2403.03952>
2. Pan, S., Li, D., Gu, H., Lu, T., Luo, X., & Gu, N. (2022). Accurate and explainable recommendation via review rationalization. In *Proceedings of the ACM Web Conference 2022* (pp. 2771–2780). ACM.
3. Ren, X., Yin, H., Chen, T., Wang, H., Huang, Z., & Zheng, K. (2021). Learning to ask appropriate questions in conversational recommendations. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 226–235). ACM.
4. An Quang Tang, Xiuzhen Zhang, Minh Ngoc Dinh, and Zhuang Li. 2025. QQSUM: A Novel Task and Model of Quantitative Query-Focused Summarization for Review-based Product Question Answering. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 20810–20831, Vienna, Austria. Association for Computational Linguistics.
5. Tanjim, M., In, Y., Chen, X., Bursztyn, V., Rossi, R. A., Kim, S., ... & Park, C. (2025, November). Disambiguation in Conversational Question Answering in the Era of LLMs and Agents: A Survey. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (pp. 9548-9561)
6. Liu, S., Gao, Y., Li, S. et al. A hallucination detection and mitigation framework for faithful text summarization using LLMs. *Sci Rep* 16, 1374 (2026). <https://doi.org/10.1038/s41598-025-31075-1>
7. McAuley Lab. (2023). Amazon Reviews 2023 dataset. [https://amazon-reviews-2023.github.io/\[amazon-rev....github.io\]](https://amazon-reviews-2023.github.io/[amazon-rev....github.io]).
8. Kim, Y. (2014). Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1746–1751). Association for Computational Linguistics. <https://doi.org/10.3115/v1/D14-1181>

Appendix

General Guidelines

1. Please use this template document to complete each deliverable assignment.
2. Each assignment must be submitted by the due date in the Course Schedule.
3. All figures should be followed by a brief description of the figure.
4. Figures can be hand-drawn and scanned in some circumstances, but the hand-drawn figure should be clear and legible to obtain full credit. Unclear hand-drawn figures will receive partial credit. For constructing figures and diagrams it is advised to use tools.
5. Figures and tables should have appropriate captions. For documenting and referencing styles please follow the APA or MLA writing style.
6. Please make sure that you provide a reference section.
7. Any material text or figure taken from books, journals or the Internet should be referenced. If you have a sentence or a figure that does not belong (authorship) to you, they must be clearly referenced. If you fail to do so your report will be considered as a case for plagiarism. It is your duty to make sure that your report is free from any activity related to plagiarism. Please see the section on Academic Integrity found below. The penalty for plagiarism will be a “0” awarded to your report. Thus, it is good to keep it simple, always have the principle to acknowledge people for their contributions.
8. Please note that the use of ChatGPT is prohibited. If the use of ChatGPT to complete the assignment has been determined, then you will receive NO credit for your project.

Project Assignment 7 (10 + 5 = 15 points)

Submit the final report after implementing all my feedback. Make sure that all the sections in the report are complete. In addition to that, the team will have an opportunity to present their project outcomes in the classroom for about 10-15 minutes. More instructions about the presentation will be provided by the instructor over the email.

Evaluation Metrics

To evaluate the model performance, several metrics are provided in each step to measure in different aspects. The metrics selection and their justification are in the table below

Table 14 : Evaluation metrics for each component

Sub-task	Metrics	Justification
Offline: Sentiment classifier	Precision, Recall, Macro F1 (per class)	Macro F1 accounts for potential class imbalance between pro and con labels in the dataset
Sub-task 1: Clarification check	Accuracy, F1-score (binary)	Binary classification; F1 accounts for class imbalance between generic and specific queries
Sub-task 2: Retrieval	Recall@K, NDCG@K	Standard IR metrics used in BLAIR paper (Hou et al., 2024); NDCG@K rewards higher-ranked relevant results
Sub-task 3: Summarization	ROUGE-1, ROUGE-2, ROUGE-L, Faithfulness score	ROUGE measures overlap with reference summaries; Faithfulness score measures proportion of summary claims grounded in retrieved evidence, directly evaluating Challenge 1

Code Base

Entire codebase can be accessed form following GitHub link.

<https://github.com/ch-supanut/A-I-574-NLP-Project>

Instruction to Run the code

It is recommended to use google Colab to run the code and create a seperate folder in the drive. The code is created to download and unzip the data, eliminating any manual hassle. Post that an individual can run notebook cells as usual.